Microstrip Model

A gentle experiment plan before simulating the SRA PCB

The idea: keep the model simple, change one thing at a time, and learn what the S-parameter plot does when the geometry or material changes.

What the current model contains

The script `run_microstrip.py` describes one straight trace above a dielectric slab and a ground plane. Port 1 sends in a pulse; port 2 measures what arrives. The model is a practice structure. It does not import the SRA PCB.

Port 1 send pulse	20 mm trace 3 mm wide	Port 2 measure arrival
	Assumed dielectric: εr = 4.2 Height: 1.6 mm	
	Ideal ground plane	

First: understand dB

The plot uses dB to express a ratio. Here the ratio is the reflected or transmitted wave compared with the wave sent into the line. A dB value is not a voltage by itself.

Plot value	Wave amplitude ratio	Power ratio (same reference impedance)
0 dB	1×: same amplitude	100%
−3 dB	about 0.71×	about 50%
−20 dB	0.1×: one tenth	1%
−40 dB	0.01×: one hundredth	0.01%

- **S11** is the reflected wave. A more negative S11 means less reflection.
- **S21** is the wave transmitted to port 2. A value closer to 0 dB means more power is transmitted.

The script normalizes the port calculations to 50 Ω. That is a reference for reporting S-parameters; it does not by itself make the physical trace exactly 50 Ω.

Before changing the design

First make sure the baseline is trustworthy. Otherwise a difference between plots could come from numerical settings instead of the change you intended.

Gate 1 — save a baseline

- Keep the present inputs: 20 mm length, 3 mm width, 1.6 mm substrate height, εr = 4.2, and 250 µm maximum mesh cell.
- Save the script inputs, the run log, and a copy of the S-parameter CSV and plot.
- The session notes say the current run reached its time-step cap before the field-decay target. Mark this first plot as **exploratory**.

Gate 2 — check the run has settled

Repeat the baseline with a larger time-step allowance, leaving geometry and mesh unchanged. Compare the S11 and S21 curves. A usable baseline should reach the solver's stopping target, or the curves should be shown to stop changing as the allowed run time increases. Record clearly if the cap is still reached.

Gate 3 — check mesh sensitivity

Once the time history is adequate, compare the present 250 µm maximum cell size with a finer mesh such as 125 µm. Keep the same geometry and materials. Compare the whole curves, especially resonances and sharp features. Finer cells increase cost and also constrain the FDTD time step, so refinement should answer a question rather than be done blindly. See the official openEMS mesh guidance.

> Do not begin the parameter sweeps until you can distinguish a real curve change from a result that changes because the run stopped too early or the mesh changed.

How to compare two runs

Keep fixed	Change	Compare
Everything else	One input only	S11 and S21 over the same frequency range
Frequency grid and port setup	Geometry or material value	Curve shape, key frequencies, and run status

For every run, use a descriptive label such as `width_2p7mm`. Keep the original CSV instead of overwriting it.

Experiments 1–4: the straight line

These experiments use the existing straight-line model. They need only small changes to the constants near the top of `run_microstrip.py`.

Experiment	Change one thing	Look for / learn
1. Trace width	Try 2.4, 3.0, and 3.6 mm. Change `TRACE_WIDTH_UM`.	A wider strip generally has lower characteristic impedance. Watch how the reflection changes. The direction and size of the S11 change depend on the port and stackup.
2. Substrate height	Try 1.3, 1.6, and 1.9 mm as sensitivity cases.	The spacing between trace and ground affects impedance and field distribution. These are learning values, not a substitute for the real board stackup.
3. Relative permittivity	Try εr = 3.8, 4.2, and 4.6 as sensitivity cases.	The electric field travels more slowly in a higher-permittivity environment, changing phase and delay. Use verified material data for any board-specific conclusion.
4. Line length	Try 10, 20, and 40 mm. Change `LINE_LENGTH_UM`.	A longer line accumulates more phase delay. In this ideal lossless model, length alone should not create physical conductor or dielectric loss.

An optional plot upgrade: phase

The current script plots only the magnitude of S11 and S21 in dB. S21 also has a phase. Plotting phase versus frequency shows how much the wave rotates as it travels along the line. From phase versus frequency, you can estimate group delay. This is a script change, so do it after the basic magnitude sweeps are understood.

> For this current model, the dielectric is lossless and the metal is ideal. Do not interpret S21 below 0 dB as a measured PCB loss without first validating the run and adding realistic material losses.

Keep a small run log

Run name	Changed input	Run reached target?	What changed in S11 / S21?
baseline	none	yes / no	starting reference
width_2p4mm	trace width	yes / no	notes
er_4p6	relative permittivity	yes / no	notes

Experiments 5–8: realistic effects

Add these only after the straight-line baseline and simple sweeps are behaving consistently. Each new feature adds a new modeling question.

Experiment	Add to the geometry/model	What it teaches
5. Losses	Finite copper conductivity and dielectric loss tangent.	S21 falls as energy is dissipated in the conductors and dielectric. Compare against the ideal-material baseline. Use actual frequency-dependent material data if available.
6. One discontinuity	Choose a width step, a 90° bend, or a small gap. Add only one.	A geometric change can reflect energy and create frequency-dependent behavior. Compare the feature against the straight line.
7. Open stub	Branch a short open-ended trace from the main line; sweep the stub length.	A stub can create a dip (notch) in S21 near a resonance. The official openEMS microstrip tutorial demonstrates this experiment.
8. Nearby trace	Add a second parallel line and a receiving port. Change spacing or parallel length one at a time.	Signal can couple from one trace into another. With port 1 active and port 2 passive the existing model gets S11 and S21; a full multiport matrix requires each port to be excited in its own run.

Field pictures

The script currently writes port results and S-parameters; it does not save a field movie or field map. Later, add a small field-dump region near the trace or discontinuity. A field view can show where energy is concentrated, but it does not replace checking convergence or S-parameters.

Useful references

- openEMS ports — port excitation, reference impedance, and S-parameter runs.
- openEMS mesh — mesh placement, resolution, and time-step cost.
- openEMS microstrip notch-filter tutorial — straight line with an open-ended stub.

> Do not add a bend, stub, losses, and a second trace all at once. If the curve changes, you would not know which addition caused it.

When to move to the SRA PCB

The microstrip model has taught you how the solver, ports, mesh, and S-parameter plots behave. The next step is one small real PCB feature, not the whole board at once.

Before modeling a real feature

- Find the fabricator's actual layer stackup, dielectric properties, and copper thickness.
- Choose one routed net or local feature with a clear question, such as a short transmission path, bend, via transition, or pair of coupled traces.
- Map only the needed copper, dielectric layers, vias, and return path into the model.
- Place ports on understandable cross-sections and confirm the reference planes.
- Repeat the convergence checks on that feature before treating its result as meaningful.

> A verified small-feature model gives a useful next milestone. A microstrip tutorial result alone does not predict the behavior of the entire SRA PCB.

Experiment worksheet

Question	Your notes
What do I expect to happen?	
Which single parameter will I change?	
Which settings must stay fixed?	
Did the solver reach its stopping target?	
What happened to S11?	
What happened to S21?	
What will I try next?	

Current script and notes

Script: `simulations/scripts/run_microstrip.py`
Current-run note: `simulations/OPENEMS_SESSION_CONTEXT.md`
The script writes geometry to `models/microstrip.xml` and S-parameter outputs under `results/`.